

Density Correction under Residue Conditioning in Primorial Systems

thinkthoughts

<https://github.com/thinkthoughts/mathematical-basis>

DRAFT v3.1

Abstract

Let P_k be the k -th primorial (squarefree, the product of the first k primes), let m be any squarefree divisor of P_k with $m > 1$, and let a be any integer with $\gcd(a, m) = 1$. We show that

$$S(P_k, L, m, a) = \{n \leq L : n \equiv a \pmod{m}, \gcd(n, P_k) = 1\}$$

departs from the naive independent-density prediction

$$\frac{\varphi(P_k)}{P_k} \frac{L}{m}$$

by an exact, computable factor

$$C(m) = \frac{m}{\varphi(m)},$$

independent of which admissible residue a is chosen, with an exact per-period count and a bounded remainder term.

This generalizes and completes an earlier draft's numerically motivated observation about the specific case $m = 6$, $a = 5$, which had reported an incorrect limiting constant of $24/25$; the corrected case gives $C(6) = 3$.

We also examine a weighted-vector construction previously labeled a correspondence to a 45° diagonal direction, find that the two objects do not even share the claimed reading, and use the discrepancy as a worked example of a three-level distinction: a shared label does not establish a shared reading, and even a shared reading would not, on its own, establish a shared specification.

1. Setup

Let

$$P_k = p_1 p_2 \cdots p_k$$

denote the k -th primorial, the product of the first k primes ($p_1 = 2, p_2 = 3, \dots$). By construction, P_k is squarefree.

Let m be a squarefree divisor of P_k with $m > 1$, and let a be an integer satisfying $\gcd(a, m) = 1$. Define

$$S(P_k, L, m, a) = \{n \leq L : n \equiv a \pmod{m}, \gcd(n, P_k) = 1\}.$$

A naive density heuristic treats the residue condition and the coprimality condition as independent: density $1/m$ from the residue class, multiplied by $\varphi(P_k)/P_k$ from coprimality to all of P_k . This gives

$$|S(P_k, L, m, a)| \approx \frac{\varphi(P_k)}{P_k} \frac{L}{m}.$$

This double-counts a restriction. Since $\gcd(a, m) = 1$, every integer satisfying $n \equiv a \pmod{m}$ is already coprime to every prime dividing m . The factor $\varphi(P_k)/P_k$ therefore re-imposes a coprimality restriction already enforced by the residue condition for primes dividing m .

2. The Density-Correction Theorem

Theorem 1 (General form). *Let P_k , m , and a be as above. Write*

$$L = qP_k + s, \quad 0 \leq s < P_k.$$

Then

$$|S(P_k, L, m, a)| = q \varphi(P_k/m) + R(s),$$

where

$$R(s) = |\{n \leq s : n \equiv a \pmod{m}, \gcd(n, P_k) = 1\}|$$

is the exact count within the partial final period and satisfies

$$0 \leq R(s) \leq \left\lceil \frac{s}{m} \right\rceil.$$

Against the naive predicted count

$$\frac{\varphi(P_k)}{P_k} \frac{L}{m},$$

the ratio of the exact count to the naive prediction converges, as $L \rightarrow \infty$, to

$$C(m) = \frac{m}{\varphi(m)},$$

independently of the admissible residue a .

Proof. Since $m \mid P_k$ and P_k is squarefree, m and P_k/m are coprime and squarefree, and P_k/m consists exactly of the prime factors of P_k that do not divide m .

Within one period $[1, P_k]$, exactly P_k/m integers satisfy

$$n \equiv a \pmod{m}.$$

By the Chinese Remainder Theorem, because m and P_k/m are coprime, the residue of n modulo P_k/m ranges uniformly over all residue classes as n ranges over its fixed class modulo m within one period.

Hence exactly the fraction

$$\prod_{p|P_k/m} \left(1 - \frac{1}{p}\right) = \frac{\varphi(P_k/m)}{P_k/m}$$

of these P_k/m values are additionally coprime to every prime dividing P_k/m . Since $\gcd(a, m) = 1$, they are already coprime to every prime dividing m . Therefore the exact count per period is

$$\frac{P_k}{m} \frac{\varphi(P_k/m)}{P_k/m} = \varphi(P_k/m).$$

The count does not depend on the specific value of a beyond $\gcd(a, m) = 1$, because the argument uses only that the residue class modulo m is fixed and coprime to m .

Using multiplicativity of Euler's totient function,

$$\varphi(P_k) = \varphi(m)\varphi(P_k/m),$$

so

$$\varphi(P_k/m) = \frac{\varphi(P_k)}{\varphi(m)}.$$

The naive predicted count per period is

$$\frac{\varphi(P_k)}{P_k} \frac{P_k}{m} = \frac{\varphi(P_k)}{m}.$$

The ratio is therefore

$$\frac{\varphi(P_k)/\varphi(m)}{\varphi(P_k)/m} = \frac{m}{\varphi(m)} = C(m).$$

This ratio is exact per full period and independent of k . Periodicity with period P_k gives the general- L statement with the bounded remainder above. $\square$

Standardized refined-density identity

Throughout this paper, the refined predicted count is written in the equivalent forms

$$E_{\text{refined}}(L) = \frac{L}{m} \prod_{p|P_k/m} \left(1 - \frac{1}{p}\right) = L \frac{\varphi(P_k/m)}{P_k}.$$

These are equal because

$$\frac{L}{m} \frac{\varphi(P_k/m)}{P_k/m} = L \frac{\varphi(P_k/m)}{P_k}.$$

Corollary 1 ($m = 6$ case). *For $m = 6$ and $a \in \{1, 5\}$,*

$$C(6) = \frac{6}{\varphi(6)} = \frac{6}{2} = 3.$$

This corrects the earlier draft's reported limiting constant $24/25$ for the case $a = 5$.

Against the refined predicted count

$$E_{\text{refined}}(L) = L \frac{\varphi(P_k/6)}{P_k},$$

the ratio is exactly 1.

Corollary 2 (Combined residue classes). *Since the per-period count $\varphi(P_k/m)$ is independent of which admissible a is used, the same ratio $C(m)$ holds for any union of admissible residue classes modulo m . For example, the classes*

$$n \equiv 1 \text{ or } 5 \pmod{6}$$

together give the same ratio 3 as either class individually, because both the exact and naive counts scale by the same number of residue classes.

Both corollaries were verified numerically at $P_k = 210$ for $m \in \{6, 10, 30\}$, multiple values of a , and L up to 10^7 , matching the predicted $C(m)$ to five decimal places or better in every case and matching the exact per-period counts $\varphi(P_k/m)$ precisely; see `tests/basis/test_persistent_constant.py`.

3. A Failed Correspondence as a Reading-Point Example

An earlier draft defined a weighted tuple

$$(9, 4, 2, 3)$$

with angles

$$\theta_k \in \{0^\circ, 60^\circ, 120^\circ, 180^\circ\}$$

and

$$V = \sum_{k=1}^4 w_k e^{i\theta_k} = 9 + 4e^{i\pi/3} + 2e^{i2\pi/3} + 3e^{i\pi} = 7 + 3\sqrt{3}i.$$

The reported reading was

$$r_V = \arg(V) \approx 36.586776^\circ.$$

The same draft separately noted that the point $(1, 1)$ has reading

$$r_D = \arg(1 + i) = 45^\circ,$$

and described the combination as a “9423 phase-lock representation,” implying a correspondence between the two constructions.

Since

$$r_V \neq r_D,$$

no such correspondence holds under the stated construction.

An exhaustive search over all 24 permutations of assigning the four weights to the four fixed angles confirms that this discrepancy is not an artifact of ordering: no permutation reaches exactly 45° , with the closest approaches at 43.898° and 46.102° (`tests/basis/test_9423_phase_lock.py`).

This gives the three-level distinction

shared label $\nRightarrow$ shared reading $\nRightarrow$ shared specification
--

The draft’s own label, “9423 phase-lock representation,” was shared across both objects by assertion, but the named readings are not shared:

$$r_V = 36.586776^\circ \neq 45^\circ = r_D.$$

The second implication is worth stating independently. Even had $r_V = r_D$ exactly, that equality alone would not establish a structural correspondence between V 's construction and the point $(1, 1)$. No transformation, projection, or limiting operation connecting the two objects would thereby be specified.

A shared label asserts a correspondence. A shared reading is, at most, evidence that two objects were run through procedures producing the same observable category. Neither, on its own, specifies the correspondence itself.

Section 2 gives a cleaner instance of the same principle. The normalized ratio for $m = 6$, $a = 5$ has taken three different numerical values during the paper's development:

$$\frac{24}{25} \quad (\text{originally reported, against no explicitly stated refined baseline}),$$

$$3 \quad (\text{against the naive predictor } (\varphi(P_k)/P_k)(L/6)),$$

and

$$1 \quad (\text{against the refined predictor } L \varphi(P_k/6)/P_k).$$

None of these numbers is meaningful without specifying which predictor produced it. The number alone is insufficient; the reading requires its producing specification to be stated alongside it.

4. Numerical Verification

At

$$P_k = 210 = 2 \cdot 3 \cdot 5 \cdot 7, \quad m = 6, \quad a = 5,$$

the numerical results are:

Table 1: Numerical verification at $P_k = 210$, $m = 6$, $a = 5$.

L	actual	naive predicted	naive ratio	refined predicted	refined ratio
10^5	11,428	3,809.52	2.99985	11,428.57	0.99995
10^6	114,285	38,095.24	2.99998	114,285.71	0.99999
10^7	1,142,856	380,952.38	3.00000	1,142,857.14	1.00000
10^8	11,428,571	3,809,523.81	3.00000	11,428,571.43	1.00000

Additional verification across

$$m \in \{6, 10, 30\}$$

and all admissible $a \bmod m$ confirms

$$C(m) = \frac{m}{\varphi(m)}$$

exactly:

$$C(6) = 3, \quad C(10) = 2.5, \quad C(30) = 3.75.$$

It also confirms the exact per-period count $\varphi(P_k/m)$, independently of a , at $P_k = 210$ and L up to 10^7 ; see `tests/basis/test_persistent_constant.py`.

5. Conclusion

The density-correction factor

$$C(m) = \frac{m}{\varphi(m)}$$

is a general, exact, elementary consequence of conditioning a coprimality count on a residue class already coprime to the modulus. It follows from the Chinese Remainder Theorem and multiplicativity of φ , requiring no sieve-theoretic or analytic machinery.

The $m = 6$ case,

$$C(6) = 3,$$

corrects an earlier draft's reported constant 24/25.

A separate weighted-vector construction claimed to correspond to a 45° diagonal direction does not do so under exhaustive search and is retained here as a worked example where a shared label fails to establish a shared reading. The corrected density theorem, meanwhile, shows why a numerical reading must be interpreted together with its producing specification.

A. Verification Code

```

from math import gcd
from sympy import totient, primefactors

def naive_predicted(limit, primorial, m):
    density = totient(primorial) / primorial
    return density * (limit / m)

def count_residue(limit, primorial, m, a):
    c = 0
    n = a
    while n <= 0:
        n += m
    for n in range(n, limit + 1, m):
        if gcd(n, primorial) == 1:
            c += 1
    return c

def refined_predicted(limit, primorial, m):
    # density = product over primes dividing Pk but NOT dividing m
    m_primes = set(primefactors(m))
    dens = 1.0
    for p in primefactors(primorial):
        if p not in m_primes:
            dens *= (1 - 1.0 / p)
    return dens * (limit / m)

```

Full runnable versions, including the general (m, a) sweep and the permutation search for Section 3, are in `tests/basis/test_persistent_constant.py` and `tests/basis/test_9423_phase_lock.py`.

B. References

- [1] G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers*, Oxford University Press. Relevant background: the Chinese Remainder Theorem, Euler’s totient function, and multiplicativity of φ .

C. Project Vocabulary

The notation in this appendix is project-specific and is not part of Theorem 1. The theorem itself remains legible and valid independently of this vocabulary.

Pi — **expand**.

$$m \mapsto \mathbb{Z}/m\mathbb{Z}.$$

This exposes the complete residue/state space for a given modulus before any restriction is imposed.

π — **extend**.

$$P_k \mapsto P_{k+1} = p_{k+1}P_k.$$

This adds another specification/constraint by extending the primorial by one further prime.

Π — **resist**.

$$\prod_{p|P_k/m} \left(1 - \frac{1}{p}\right).$$

This is the fraction of candidates remaining admissible under the divisibility constraints not already imposed by the residue condition; it is the quantity appearing directly in the proof of Theorem 1.

These roles give the progression

$$\text{Pi (expand)} \longrightarrow \pi \text{ (extend)} \longrightarrow \Pi \text{ (resist)} \longrightarrow r \text{ (read)}.$$

These names label roles already present in the proof of Theorem 1. The proof itself does not depend on this vocabulary and holds independently of it.